

CSE222 HW7 REPORT

MUHAMMET FURKAN YILGIN

220104004964

AVL Tree Implementation

First of all, I would like to show the implementation of my AVLTree using the screenshots of my code below.

Balancing the tree :

```
/**
 * Method to rebalance the tree according to the current height and balance values.
 * According to the current balance value and left/right subtree values , tree is balanced using ll,lr,rr or rl rotations.
 * @param node
 * @return node
 */
private Node rebalanceAVLTree(Node node){
    HeightUpdate(node);

    int current_balance = getBalance(node);

    if (current_balance > 1) {
        if (getBalance(node.left) >= 0) {
            return rightRotate(node);
        } else {
            node.left = leftRotate(node.left);
            return rightRotate(node);
        }
    } else if (current_balance < -1) {
        if (getBalance(node.right) <= 0) {
            return leftRotate(node);
        } else {
            node.right = rightRotate(node.right);
            return leftRotate(node);
        }
    }

    return node;
}
```

Add :

```
public void addNode(Stack newStock) {
    root = addNode(root, newStock);
}

/**
 * The add method of AVLTree.It checks the symbol of the stock to find where to add the stock.If symbol is smaller goes to left subtrees,right otherwise
 * However if it equals it updates the stock value of that symbol with the new one.
 * @param newNode node to be added.
 * @param newStock stock to be added.
 * @return updated node.
 */
private Node addNode(Node newNode,Stack newStock){
    if(newNode == null )
        return new Node(newStock);

    int symbol_value = newStock.getSymbol().compareTo(newNode.stock.getSymbol());

    if(symbol_value == 0){
        newNode.stock = newStock;
        return newNode;
    }
    else if(symbol_value < 0)
        newNode.left = addNode(newNode.left,newStock);

    else if(symbol_value > 0)
        newNode.right = addNode(newNode.right,newStock);

    // 2. Update height of this ancestor node

    // Update height of this ancestor node

    newNode.height = 1 + getBigger(getHeight(newNode.left),getHeight(newNode.right)) ;

    return rebalanceAVLTree(newNode) ;
}
```

Remove :

```
public void removeNode(String symbol){
    root = removeNode(root,symbol);
}

/**
 * Method for removing nodes in tree.It searches the node to be removed then after it is found
 * it removes it according to its left,right or both subtrees.After the removal it updates the height of the node
 * and rebalances it.
 * @param node
 * @param symbol
 * @return
 */
private Node removeNode(Node node, String symbol) {
    if (node == null) {
        return null;
    }

    int symbolValue = symbol.compareTo(node.stock.getSymbol());

    // If the symbol to be deleted is smaller than the current node's symbol,
    // then it goes to the left subtree
    if (symbolValue < 0) {
        node.left = removeNode(node.left, symbol);
    }
    // If the symbol to be deleted is greater than the current node's symbol,
    // then it goes to the right subtree
    else if (symbolValue > 0) {
        node.right = removeNode(node.right, symbol);
    }
    // If symbol is same as the current node's symbol, then this is the node to be deleted
    else {
        // Node with only one child or no child
        if (node.left == null) {
            return node.right;
        } else if (node.right == null) {
            return node.left;
        }

        // Node with two children: Get the inorder successor (smallest in the right subtree)
        Node temp = minValueNode(node.right);

        // Copy the inorder successor's data to this node
        node.stock = temp.stock;

        // Delete the inorder successor
        node.right = removeNode(node.right, temp.stock.getSymbol());
    }

    // Update the height of the current node
    node.height = getBigger(getHeight(node.left), getHeight(node.right)) + 1;

    // return the rebalanced tree.
    return rebalanceAVLTree(node);
}
```

Search :

```
/**
 * The search method that searches the tree according to the given symbol.Goes left if the symbol value is smaller , right if bigger
 * returns the node if it 0 which means it is found.Returns null if it is not found or the node is null.
 * @param node
 * @param symbol
 * @return
 */
private Node searchNode(Node node , String symbol){
    if(node == null) return null ;
    int symbol_value = symbol.compareTo(node.stock.getSymbol());
    if(symbol_value==0)
        return node;
    else if(symbol_value < 0){
        return searchNode(node.left, symbol);
    }
    else {
        return searchNode(node.right, symbol);
    }
}

/**
 * The search method of AVLTree.
 * @param symbol
 * @return
 */
public Stock searchNode(String symbol){
    Node searchedNode = searchNode(root,symbol);
    if(searchedNode == null) return null ;

    return searchedNode.stock ;
}
```

Traversals :

```
/**
 * Method for PreOrderTraversal.It first prints the symbol of the root then goes to left and right subtrees respectively.
 * @param node
 */
private void PreOrderTraversal(Node node){
    if(node==null)
        return;
    System.out.println(node.stock.printStockInfo());
    PreOrderTraversal(node.left);
    PreOrderTraversal(node.right);
}

public void InOrderTraversal(){
    System.out.println(x: " --- IN ORDER TRAVERSAL ---");
    InOrderTraversal(root);
}

/**
 * Method for InOrderTraversal.It first goes to left subtrees , prints the symbol then goes to right subtree.
 * @param node
 */
private void InOrderTraversal(Node node){
    if(node==null)
        return;
    InOrderTraversal(node.left);
    System.out.println(node.stock.printStockInfo());
    InOrderTraversal(node.right);
}

/**
 * Method for PostOrderTraversal.It goes left and right subtrees and prints the symbol.
 * @param node
 */
private void PostOrderTraversal(Node node){
    if(node==null)
        return;
    PostOrderTraversal(node.left);
    PostOrderTraversal(node.right);
    System.out.println(node.stock.printStockInfo());
}
```

One Operation Only :

Mixed Operations :

```
public void ScriptToFileMixed(int size){
    try {
        FileWriter writer = new FileWriter(fileName:"MixedOperationTestAVL.txt") } {
            Random random = new Random();
            double max = 999.99;
            double min = 0.0;
            long maxLong = 999999999;
            long minLong = 9999;
            String operations[] = {"ADD", "REMOVE", "UPDATE", "SEARCH"};
            for(int i=0; i<size; i++){
                int choice = random.nextInt(bound:4);
                switch (operations[choice]) {
                    case "ADD":
                        double price = min + (max - min) * random.nextDouble();
                        long volume = minLong + (long)((maxLong - minLong) * random.nextDouble());
                        long marketCap = minLong + (long)((maxLong - minLong) * random.nextDouble());
                        writer.write(String.format(format:"%s SYMBOL%d %.2f %d %d\n", operations[choice], i, price, volume, marketCap));
                        break;
                    case "REMOVE":
                        writer.write(String.format(format:"%s SYMBOL%d\n", operations[choice], i));
                        break;
                    case "UPDATE":
                        price = min + (max - min) * random.nextDouble();
                        volume = minLong + (long)((maxLong - minLong) * random.nextDouble());
                        marketCap = minLong + (long)((maxLong - minLong) * random.nextDouble());
                        writer.write(String.format(format:"%s SYMBOL%d NEW%d %.2f %d %d\n", operations[choice], i, i, price, volume, marketCap));
                        break;
                    case "SEARCH":
                        writer.write(String.format(format:"%s SYMBOL%d\n", operations[choice], i));
                }
            }
        }
    }
}
```

Performance Analysis Implementation

```
public long performPerformanceAnalysis(StockDataManager manager, int size, String operation) {
    long startTime = 0;
    long endTime = 0;
    Random random = new Random();
    if(operation.equals(anObject:"MIXED")){
        String operations[] = {"ADD", "REMOVE", "UPDATE", "SEARCH"};
        int choice = random.nextInt(bound:4);
        operation = operations[choice];
    }
    //System.out.println(operation);
    switch(operation){
        case "ADD":
            startTime = System.nanoTime();
            for (int i = 0; i < size; i++) {
                manager.addOrUpdateStock("SYM" + i, Math.random() * 100, (long) (Math.random() * 1000000), (long) (Math.random() * 1000000000));
            }
            endTime = System.nanoTime();
            break;
        case "REMOVE":
            startTime = System.nanoTime();
            for (int i = 0; i < size; i++) {
                manager.removeStock("SYM" + i);
            }
            endTime = System.nanoTime();
            break;
        case "UPDATE":
            startTime = System.nanoTime();
            for (int i = 0; i < size; i++) {
                manager.addOrUpdateStock("SYM" + i, Math.random() * 100, (long) (Math.random() * 1000000), (long) (Math.random() * 1000000000));
            }
            endTime = System.nanoTime();
            break;
        case "SEARCH":
            startTime = System.nanoTime();
    }
```

- Calculates the time taken for the given size and operation and returns it. This time then will be used for drawing the graph.
- For the mixed operation it chooses one of the operations each time it is called and returns the time according to that chosen operation time.

Main and Design Choices

```
Run | Debug
public static void main(String[] args){
    StockDataManager sdm = new StockDataManager();

    System.out.println(x:"Graph is being plotted...");
    SwingUtilities.invokeLater(() -> {
        String plotType = "line"; // Change to "scatter" for scatter plot
        GUIVisualization frame = new GUIVisualization(plotType, size:1000, operation:"ADD"); // Create a new instance of GUIVisualization
        frame.setVisible(b:true); // Make the frame visible // NOTE : this " " part should be changed for the operation choice of the graph.
    }); // ADD/REMOVE/UPDATE/SEARCH/MIXED

    //sdm.ScriptToFile(100); // to write to the file only 1 operation 100 times.

    if (args.length != 1) { // if no filename is entered in command line
        System.out.println(x:"Usage: java StockDataManager <input_file>");
        sdm.ScriptToFileMixed(size:50); // to write to the file only mixed operations 50 times.
        sdm.InputFormat(fileName:"MixedOperationTestAVL.txt");
        return;
    }

    String inputFile = args[0]; // if file name is entered by user in command line 'java DataStockManager' 'filename'

    sdm.InputFormat(inputFile); // trying the operations in the file which was the entered by user in the command line.
}
```


A screenshot of the main method is above. If the program has been run using makefile the scripted file with mixed operations will be created first and then it will be used with the operation in it then the AVLTree will be printed.

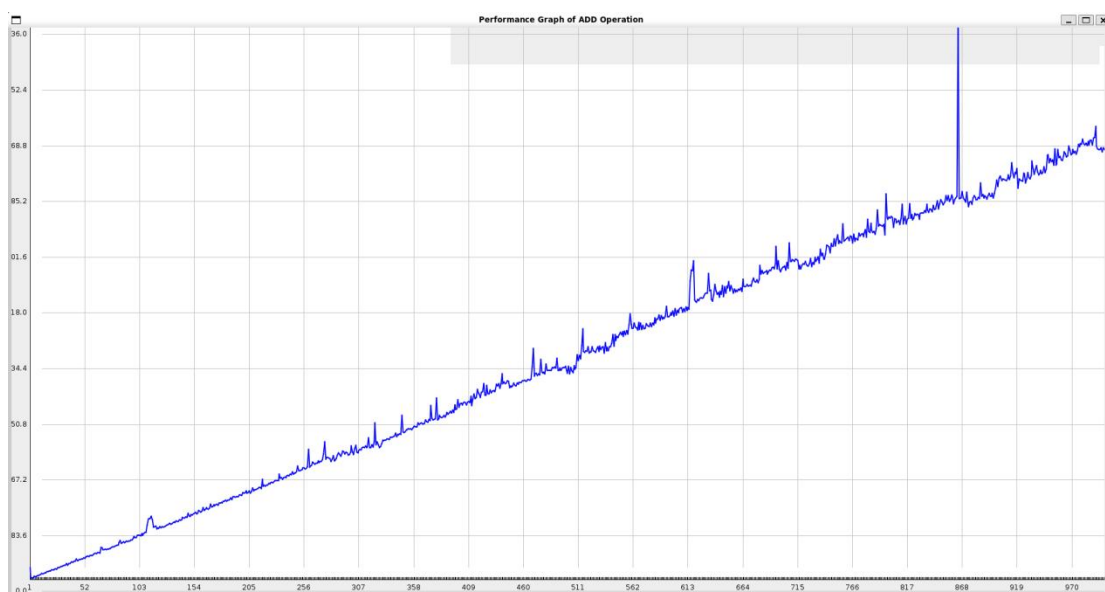
However, if the program has been started with the command 'java StockDataManager 'filename' ' AVLTree will be created according to that file and will be printed. For both running styles the graph will be created and showed for add operation only too. For other operations it must be changed from main method line 25 to other operations(remove,update,search or mixed).

It is important to note that when it is run with 'java StockDataManager filename' the graph is plotted more differently than the photos below which were plotted when it was run by make command.

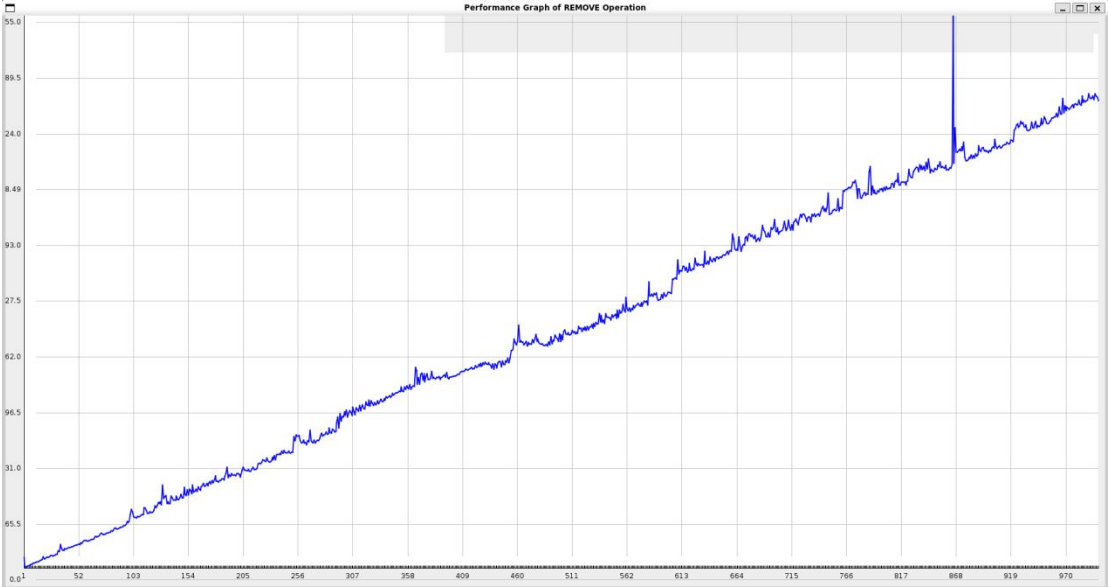
Shortly , these are my implementations of the AVLTree , my main method and design choices. The graph photos for every operations are below.

Performance Analysis Graphs

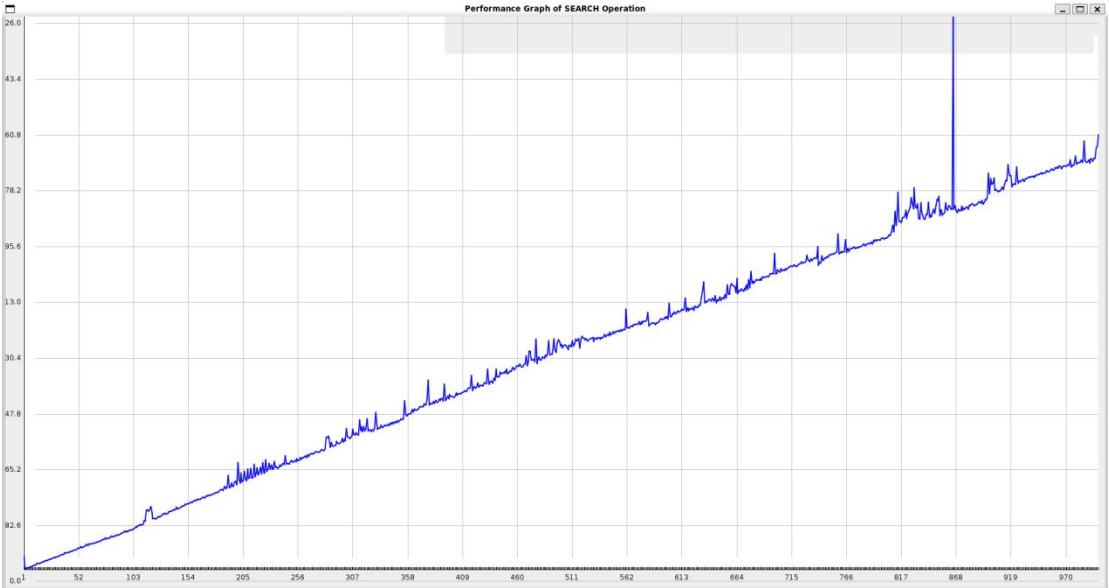
Graph of Add Operation :



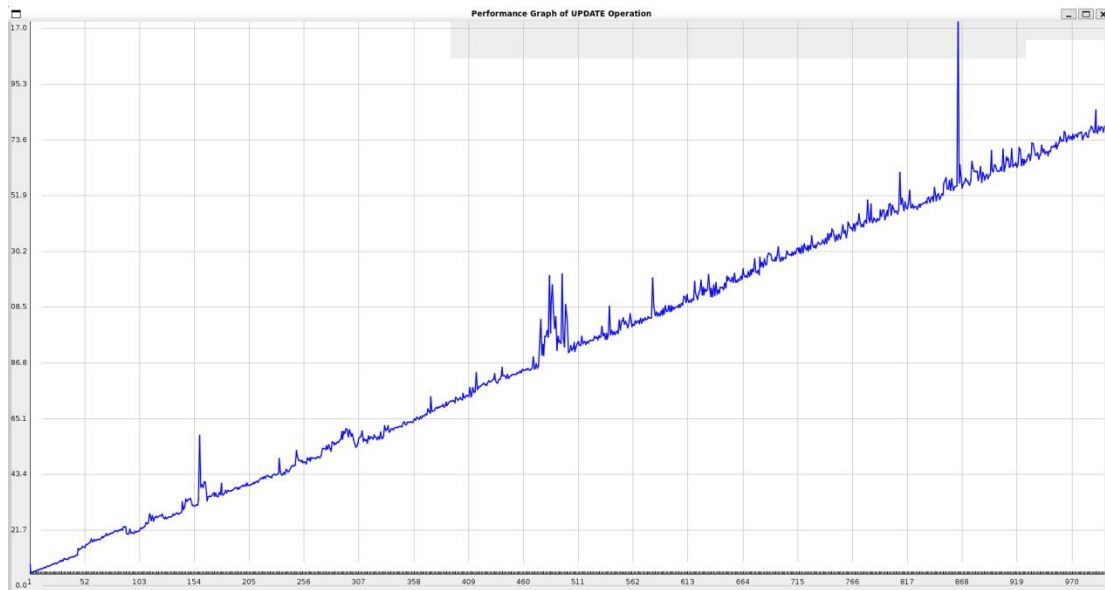
Graph of Remove Operation :



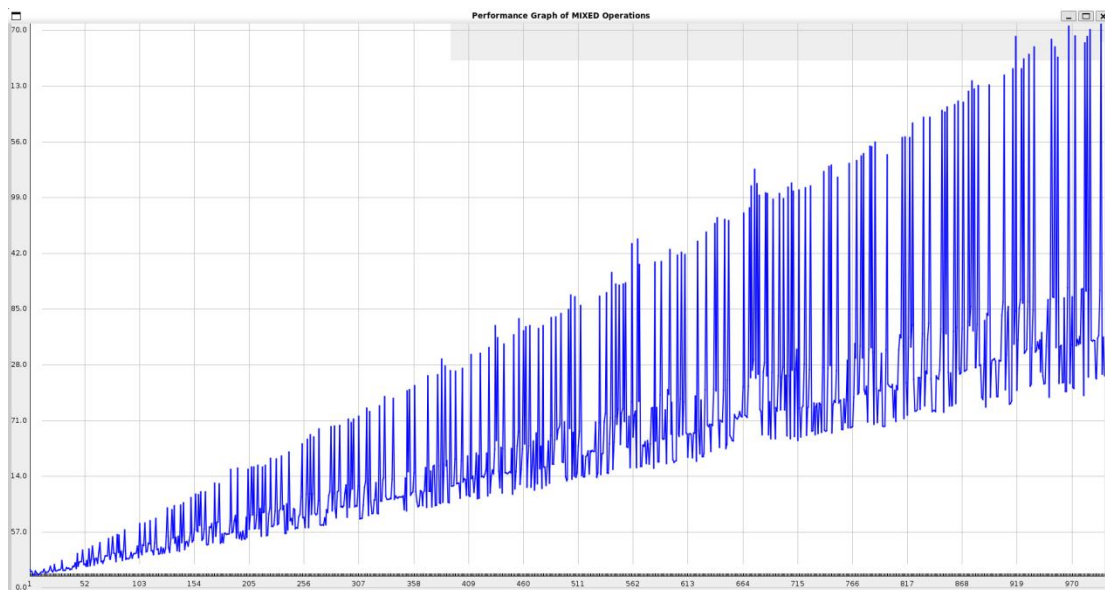
Graph of Search Operation :



Graph of Update Operation :



Graph of Mixed(Add,Remove,Update,Search) Operations :



- Overall , these are the graphs of my AVLTree even though I tried to make them look as closer to $\log n$ as possible for some of the values and operations it did not happen. However , as it can be seen from my implementations from the previous pages my implementations are $\log$ time complexity.

EXAMPLE OUTPUTS

```
mfurkan@DESKTOP-625529P:/mnt/c/Users/mfurk/Desktop/VSC Codes/CSE222-HW7$ make
rm -f *.class
javac -d . AVLTree.java Stock.java StockDataManager.java GUIVisualization.java
java -Xint StockDataManager
Graph is being plotted...
Usage: java StockDataManager <input_file>
--- PRE ORDER TRAVERSAL ---
Symbol -> SYMBOL24 Price -> 94.69 Volume -> 381948311 MarketCap -> 608640296
Symbol -> SYMBOL2 Price -> 675.26 Volume -> 699921281 MarketCap -> 144175901
Symbol -> SYMBOL10 Price -> 811.23 Volume -> 90452089 MarketCap -> 448571264
Symbol -> SYMBOL13 Price -> 389.6 Volume -> 346193462 MarketCap -> 871064249
Symbol -> SYMBOL20 Price -> 576.06 Volume -> 644318599 MarketCap -> 902363696
Symbol -> SYMBOL41 Price -> 156.76 Volume -> 844252486 MarketCap -> 459451096
Symbol -> SYMBOL29 Price -> 516.1 Volume -> 679680874 MarketCap -> 584705694
Symbol -> SYMBOL5 Price -> 625.36 Volume -> 793381144 MarketCap -> 339218574
Symbol -> SYMBOL43 Price -> 482.69 Volume -> 681358035 MarketCap -> 293903514
Symbol -> SYMBOL6 Price -> 624.38 Volume -> 273113669 MarketCap -> 837535942
```

```
mfurkan@DESKTOP-625529P:/mnt/c/Users/mfurk/Desktop/VSC Codes/CSE222-HW7$ java StockDataManager Stock2.txt
Graph is being plotted...
--- PRE ORDER TRAVERSAL ---
Symbol -> MNO Price -> 12.8 Volume -> 600000 MarketCap -> 3000000
Symbol -> DEF Price -> 15.75 Volume -> 750000 MarketCap -> 3750000
Symbol -> ABC Price -> 10.25 Volume -> 500000 MarketCap -> 2500000
Symbol -> GHI Price -> 20.0 Volume -> 1250000 MarketCap -> 6250000
Symbol -> XYZ Price -> 25,5 Volume -> 1000000 MarketCap -> 5000000
mfurkan@DESKTOP-625529P:/mnt/c/Users/mfurk/Desktop/VSC Codes/CSE222-HW7$
```

Important Note : When I tried the program to run in Ubuntu for ‘java StockDataManager filename’ command it somehow does not read the lines in files containing ADD XYZ ‘25.50’ 1000000 5000000 instead of that it reads it when the double has comma between them like ADD XYZ ‘25,50’ 1000000 5000000. I am not sure what is the reason behind it and maybe it is because of the Ubuntu version of mine.

- These are the example outputs for both running style of the code(make and java with filename). Thanks for your time. Have a nice day!